



LangGraph & Agentic Orchestration

Day 08 · State Machines cho Agents · 2h theory + 2h guided lab

Instructor

VinUniversity · Phase 2 · Track 3 · Week 5

HÃY SUY NGHĨ...

“Khi agent cần loop, retry, human approval và resume sau crash, chain một chiều còn đủ không?”

Langgraph when Linear Routing is not Enough
when Retry is necessary for you Use Case

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Mục tiêu & lịch học
2. Khi nào chain không đủ?
3. Core API
4. Persistence & Time Travel
5. Human-in-the-Loop & Error Recovery
6. Lab 4 giờ
7. Takeaways

Mục tiêu & lịch học

2 giờ lý thuyết cô đọng, 2 giờ lab có hướng dẫn; bài lab thiết kế đủ 4 giờ để phân loại năng lực.

01

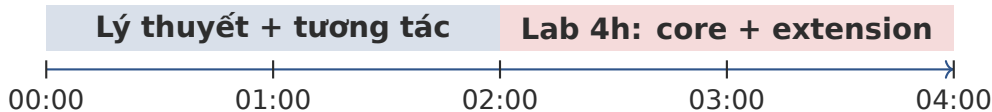
Conceptual outcomes

- Phân biệt LCEL chain, agent loop và stateful graph.
- Thiết kế state, node, edge, reducer trong LangGraph.
- Hiểu checkpointing, time travel, HITL và error recovery.

Practical outcomes

- Xây dựng workflow có conditional routing, retry và interrupt.
- Ghi trace/metric phục vụ chấm điểm.
- Viết report kỹ thuật ngắn theo rubric production.

Checkpoint: Cuối buổi: mỗi nhóm demo một graph chạy được trên test case cơ bản; học viên giải hoàn thiện thêm crash recovery và report.



2h lý thuyết

- 20' LCEL gap + state machine
- 30' StateGraph API
- 25' persistence + checkpointing
- 25' HITL + error recovery
- 20' metric/report briefing

2h trên lớp + 2h mở rộng

- 0-2h: build runnable core graph
- 2-3h: persistence + crash-resume
- 3-4h: metrics, report, polish

Khi nào chain không đủ?

LCEL phù hợp pipeline một chiều; agent production thường cần trạng thái, nhánh, vòng lặp và kiểm soát lỗi.

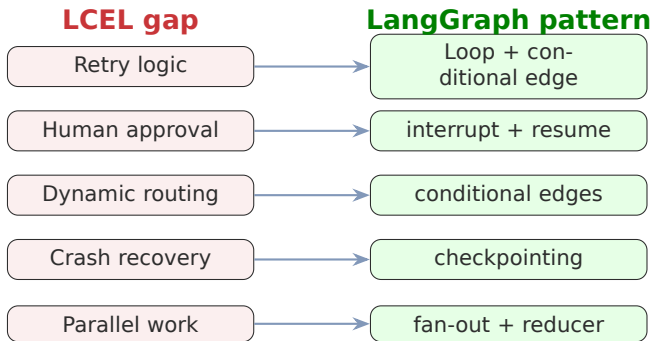


khó loop lại
khó pause cho human
khó resume sau crash

Chain đủ khi:

- task đơn giản, single-shot;
- không cần retry thông minh;
- không cần human approval;
- không cần lưu state dài hạn.

Workflow của bạn có cần quyết định bước tiếp theo dựa trên kết quả bước trước không? Nếu có, hãy nghĩ tới graph.



Mini poll - 4 phút

Trong sản phẩm bạn từng làm, vấn đề nào xuất hiện nhiều nhất: retry, routing, human approval, crash recovery hay parallel work?

LangGraph — Framework orchestration theo graph: **typed state** + **node functions** + **edges/conditional edges** + **checkpointing** để xây workflow agent có loop, interrupt, persistence và fault tolerance.

Khi dùng

- agent cần nhiều bước và quyết định động;
- cần human-in-the-loop;
- cần khôi phục sau lỗi;
- cần trace/debug.

docs.langchain.com/oss/python/langgraph/overview

Khi chưa cần

- prompt đơn lẻ;
- ETL tuyến tính;
- không có state;
- không cần approval hoặc audit.

Flow

1. Nhận ticket.
2. Classify: billing, bug, policy, urgent.
3. Nếu thiếu thông tin: hỏi lại khách.
4. Nếu rủi ro cao: dừng để human approve.
5. Nếu tool lỗi: retry hoặc dead-letter.

Routing, loop hỏi lại, HITL và retry đều phụ thuộc state hiện tại. Một chain tuyến tính sẽ nhanh trở nên khó maintain.

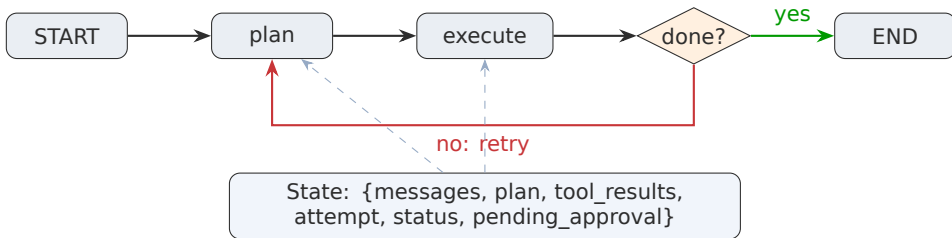
Think-pair-share - 5 phút

Viết 1 state field cần có cho ticket workflow và lý do.

Core API

State, node, edge, reducer là bốn khái niệm nền tảng của StateGraph.

03



Python function đọc state và trả về partial update.

Đường chuyển bước; có thể cố định hoặc conditional.

```
from typing import Annotated, TypedDict
from operator import add

class AgentState(TypedDict):
    messages: Annotated[list[str], add]
    query: str
    route: str
    attempt: int
    tool_results: Annotated[list[str], add]
    final_answer: str | None
    errors: Annotated[list[str], add]
```

5 quy tắc thiết kế state

1. Flat, ít nested dict.
2. Reducer rõ cho list.
3. Typed và validate được.
4. Lean: không lưu blob lớn.
5. Versioned khi schema thay đổi.

Lưu ý: Default reducer là overwrite. Nếu 2 node cùng ghi một field list mà không khai báo reducer, rất dễ mất dữ liệu.

Overwrite phù hợp cho

- status hiện tại;
- route hiện tại;
- final answer;
- counter nếu chỉ một node ghi.

Append phù hợp cho

- messages;
- tool results;
- errors;
- audit events;
- metric records.

Quick check - 3 phút

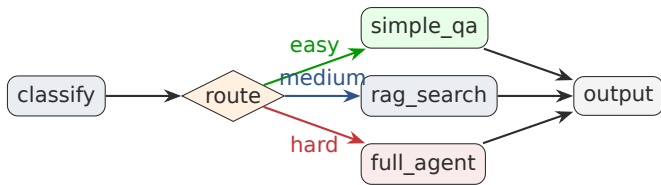
Field `audit_log` nên overwrite hay append? Vì sao?

```
def classify_node(state: AgentState) -> dict:
    # TODO(student): implement routing policy
    route = classify_query(state["query"])
    return {
        "route": route,
        "messages": [f"classified:{route}"],
    }

def tool_node(state: AgentState) -> dict:
    # Nodes should be small and testable
    result = run_tool(state["query"])
    return {"tool_results": [result]}
```

Checklist

- Pure-ish: không side effect nếu tránh được.
- Idempotent cho retry.
- Return partial update, không mutate toàn state.
- Log đủ cho audit.
- Timeout và error typed.



Nhận state, trả về tên nhánh tiếp theo. Dùng để tối ưu cost, latency và risk.

- Easy query: cheap path.
- Missing info: ask user.
- Risky action: approval.
- Repeated error: fallback/dead-letter.

```
from langgraph.graph import StateGraph, START, END

graph = StateGraph(AgentState)
graph.add_node("classify", classify_node)
graph.add_node("answer", answer_node)
graph.add_node("tool", tool_node)

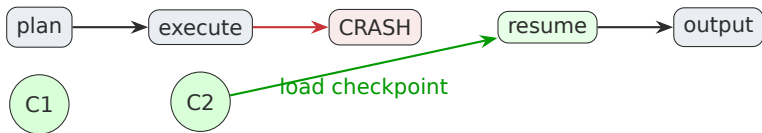
graph.add_edge(START, "classify")
graph.add_conditional_edges(
    "classify", route_next,
    {"simple": "answer", "tool": "tool"},
)
graph.add_edge("answer", END)
compiled = graph.compile(checkpointer=saver)
```

Build order

1. Define state schema.
2. Implement nodes.
3. Implement route functions.
4. Add edges.
5. Compile with checkpointer.
6. Invoke with thread id.

Persistence & Time Travel

Checkpointing biến graph thành workflow có thể pause, resume, replay và debug.



Memory saver:
nhanh, không bền
sau restart.

SQLite saver: persis-
tent, dễ demo.

Postgres saver: phù
hợp service nhiều
thread.

Lưu ý: Large state = checkpoint lớn = chậm. Lưu references thay vì full document/blob.

- **Thread**: một phiên workflow, ví dụ một user request hoặc một ticket.
- **Checkpoint**: snapshot state sau mỗi super-step khi graph có checkpointer.
- **Replay**: chạy lại từ một checkpoint để debug hoặc A/B test route khác.
- **Update state**: chỉnh state tại checkpoint trước khi resume, hữu ích cho HITL.

Khi khách báo “agent gửi sai email”, bạn cần state history để biết node nào quyết định sai, input lúc đó là gì, human đã approve hay chưa.

docs.langchain.com/oss/python/langgraph/persistence

```
config = {"configurable": {"thread_id": "ticket-123"}}

result = compiled.invoke(
    {"query": "Refund request for order 42"},
    config=config,
)

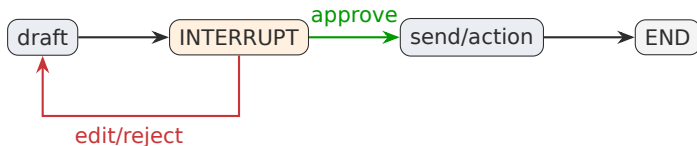
snapshot = compiled.get_state(config)
history = list(compiled.get_state_history(config))
```

Lab metric liên quan

- Có thread id riêng cho mỗi run.
- Có state history sau run.
- Có trace events đủ để tính node count, retry count, approval count.

Human-in-the-Loop & Error Recovery

Agent production phải biết khi nào tự làm, khi nào hỏi người, khi nào dừng an toàn.



Approval trước destructive action; clarification khi thiếu thông tin; escalation khi vượt quyền; review trước publish.

Graph pause, lưu state; human trả lời; graph resume từ đúng vị trí với state mới.

Role-play - 6 phút

Một bạn đóng agent, một bạn đóng reviewer. Reviewer chỉ được approve khi state có đủ evidence.

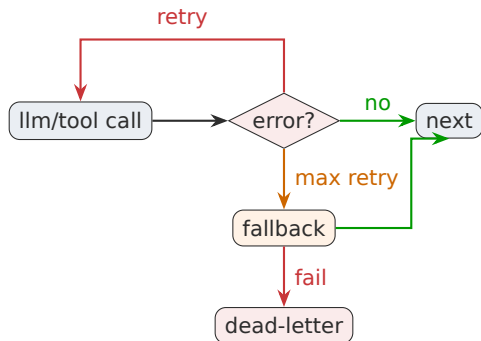

```
from langgraph.types import interrupt, Command

def approval_node(state: AgentState) -> dict:
    decision = interrupt({
        "action": state["proposed_action"],
        "risk": state["risk_level"],
        "evidence": state["tool_results"],
    })
    return {"approval": decision}

# Resume later:
compiled.invoke(Command(resume={"approved": True}), config)
```

Chăm điểm lab

- Có interrupt object rõ ràng.
- Có route approve/reject/edit.
- Report ghi số lần approval và kết quả.
- Không execute destructive action khi chưa approve.



3 tầng

1. Retry với backoff và max attempts.
2. Fallback model/tool.
3. Dead-letter để manual review.

Lưu ý: Node retry phải idempotent. Gửi email, charge payment, update database cần idempotency key.

Metrics bắt buộc

- task success rate;
- nodes visited;
- retry count;
- interrupt count;
- state validation errors;
- latency per run;
- resume success.

Report bắt buộc

- architecture diagram;
- state schema;
- test cases;
- metrics table;
- failure analysis;
- improvement plan.

Checkpoint: Lab sẽ chấm bằng cả code chạy được, metrics JSON và report markdown.

Lab 4 giờ

Xây LangGraph workflow cho agent xử lý yêu cầu support có routing, HITL, retry và metric report.

06

- Hoàn thiện production skeleton repo: state schema, nodes, graph wiring, persistence adapter.
- Chạy 6 test scenarios: simple, tool, missing-info, risky-action, transient-error, max-error.
- Xuất file metrics JSON và report markdown theo template.
- Học viên giải hoàn thành extension: crash-resume, time-travel debug hoặc parallel fan-out.

Skeleton đã có vùng `TODO(student)`. Không cần viết lại kiến trúc repo; tập trung hoàn thiện logic và bằng chứng chấm điểm.

Thời gian	Việc cần làm	Deliverable
0-30'	Setup repo, chạy tests baseline, đọc state schema	screenshot/tests log
30-75'	Implement core nodes + graph wiring	core tests pass
75-120'	Conditional routing + retry + HITL mock	6 scenarios run
120-180'	Persistence/checkpoint + crash-resume extension	trace JSON/history
180-225'	Metrics runner + report template	metrics.json + re- port.md
225-240'	Demo, cleanup, self-assessment	final zip/repo

Hạng mục	Điểm	Tiêu chí
Architecture & state	20	Typed state, reducer đúng, node nhỏ và testable
Graph behavior	25	Routing đúng, retry có giới hạn, HITL hoạt động
Persistence & recovery	15	Checkpoint, thread id, resume hoặc mock tương đương
Metrics & tests	20	Metrics JSON hợp lệ, 6 scenarios, tests pass
Report & demo	15	Report rõ, failure analysis, diagram/screenshot
Production hygiene	5	README, config, typing, lint, env handling

1. Graph của bạn có những node nào và state field quan trọng nhất là gì?
2. Một test case đi qua route nào? Có retry/interrupt không?
3. Metrics JSON cho thấy success rate, retry count, interrupt count là bao nhiêu?
4. Bạn đã chứng minh resume/crash recovery thế nào?
5. Nếu thêm 1 ngày, bạn sẽ productionize phần nào trước?

Takeaways

LangGraph không chỉ là thư viện orchestration; nó là cách thiết kế agent như một system có state, audit và recovery.

07

Những ý chính cần nhớ trước khi sang bài tiếp theo

- Dùng LCEL cho pipeline tuyến tính; dùng LangGraph khi có loop, conditional route, persistence hoặc HITL.
- State schema và reducer quyết định độ ổn định của graph.
- Checkpointing là nền tảng cho HITL, memory, time travel và fault tolerance.
- Production agent cần metric, trace và report, không chỉ demo chạy được.

1. LangGraph documentation: Persistence, Human-in-the-loop, Functional API.
docs.langchain.com/oss/python/langgraph
2. LangGraph reference: StateGraph, interrupt, Command, checkpointers. reference.langchain.com
3. LangChain blog/docs examples for agent workflows and deployment. langchain.com

Hỏi & Đáp
